Secure Code Review : VulnNotes (Flask)

Task 3 — Cybersecurity Internship Auditor: Douaa **Date:** May 2026 **Application:** VulnNotes — Python/Flask notes web application (~160 LOC) **Methodology:** Static analysis (Bandit, Semgrep), dependency scanning (pip-audit), manual code review

---

1. Executive Summary

A security review of the VulnNotes Flask application was conducted using a combination of three static analysis tools and manual code inspection. The audit uncovered **15 unique application-layer vulnerabilities** and **27 known CVEs in third-party dependencies**, for a total of **42 distinct issues**. Severities range from **Low** (debug mode left on) to **Critical** (SQL injection in authentication, remote code execution via insecure deserialization, OS command injection).

The application is **not safe for production use** in its current form. Multiple findings can be chained to achieve full account takeover and remote code execution.

Severity	Application Findings	Dependency CVEs
Critical	4	3
High	5	11
Medium	4	10
Low	2	3
Total	**15**	**27**

---

2. Scope & Methodology

The review covered `app.py` (the Flask application) and its Jinja2 templates (`templates/*.html`). External dependencies pinned in `requirements.txt` were also audited.

Three complementary tools were used, each catching issues the others miss:

Tool	Purpose	Findings
Bandit	Python AST-based SAST	13
Semgrep	Multi-framework pattern matcher (304 rules run)	31
pip-audit	PyPI Advisory Database CVE check	27
Manual review	Logic flaws & authorization issues	3

After deduplication (e.g., the same SQL injection line flagged by 3 separate Semgrep rules), **15 unique application vulnerabilities** were confirmed.

---

3. Critical & High-Severity Findings

F-01 — SQL Injection in Authentication (Critical) — CWE-89

Location: `app.py`, lines 84–88 (`/login`); also `/notes` and `/note/<id>` **Detected by:** Bandit (`B608`×3), Semgrep (`tainted-sql-string` rules ×3)

User-controlled input is concatenated directly into SQL queries:

query = "SELECT * FROM users WHERE username = '" + username + "' AND password = '" + pwd_hash + "'"

Impact: Authentication bypass and full database read/write. Verified by entering username `' OR '1'='1' --` and any password — login succeeds as the first user.

Remediation: Use parameterized queries: `db.execute("SELECT * FROM users WHERE username = ? AND password = ?", (username, pwd_hash))`.

F-02 — OS Command Injection in `/ping` (Critical) — CWE-78

Location: `app.py`, line 137 **Detected by:** Bandit (`B602`), Semgrep (`subprocess-injection`, `subprocess-shell-true`)

result = subprocess.check_output('ping -c 1 ' + host, shell=True)

Impact: Remote code execution as the Flask process user. `?host=localhost;id` runs arbitrary commands.

Remediation: Pass arguments as a list with `shell=False`, and validate the host against a strict pattern (`re.match(r'^[a-zA-Z0-9.-]+$', host)`).

F-03 — Insecure Deserialization in `/restore` (Critical) — CWE-502

Location: `app.py`, line 155 **Detected by:** Bandit (`B301`, `B403`), Semgrep (`insecure-deserialization`, `pickle.avoid-pickle`)

obj = pickle.loads(base64.b64decode(blob))

Impact: Remote code execution. A crafted pickle payload runs arbitrary Python on the server.

Remediation: Replace `pickle` with a safe serializer such as JSON. If structured data is required, sign payloads with `itsdangerous`.

F-04 — Cross-Site Scripting via `Markup()` (High) — CWE-79

Location: `app.py` line 110 + `templates/notes.html` (`|safe` filter) **Detected by:** Bandit (`B704`×2), Semgrep (`explicit-unescape-with-markup`, `raw-html-format`)

User-supplied note titles and bodies are wrapped in `Markup()` (which marks them safe) and rendered with `|safe`, bypassing Jinja2's auto-escaping.

Impact: Stored XSS — session cookie theft (worsened by F-10), keylogging, account takeover.

Remediation: Remove `Markup()` and `|safe`; let Jinja2 escape by default. If HTML is genuinely required, sanitize with `bleach` against an allowlist.

F-05 — Server-Side Request Forgery in `/preview` (High) — CWE-918

Location: `app.py`, line 149 **Detected by:** Semgrep only (`ssrf-requests`, `ssrf-injection-requests`) — *Bandit missed this entirely*

```
url = request.args.get('url', '')
r = requests.get(url, timeout=5)
```

Impact: Attacker can pivot to internal services, read cloud metadata endpoints (`http://169.254.169.254/`), and scan the internal network from the server.

Remediation: Parse the URL, resolve the hostname, and reject private/loopback/link-local IP ranges before issuing the request. Maintain a strict allowlist of permitted domains.

F-06 — Insecure Direct Object Reference in `/note/<id>` (High) — CWE-639

Location: `app.py`, lines 116–122 **Detected by: Manual review only** — no scanner flagged this

The endpoint checks that *a* user is logged in but never verifies that the note belongs to them:

```
cur = get_db().execute("SELECT * FROM notes WHERE id=" + note_id)
```

Impact: Alice can read Bob's notes by visiting `/note/2`. Horizontal privilege escalation.

Remediation: Add `AND owner = ?` to the query, passing `session['user']` as a parameter.

F-07 — Weak Password Hashing (MD5) (High) — CWE-916

Location: `app.py`, lines 62, 64, 83 **Detected by:** Bandit (`B324`×3), Semgrep (`md5-used-as-password`)

```
hashlib.md5(password.encode()).hexdigest()
```

Impact: MD5 is broken for password hashing — unsalted MD5 of common passwords can be reversed via rainbow tables in seconds.

Remediation: Use `bcrypt`, `argon2-cffi`, or `hashlib.scrypt` with per-user salts.

F-08 — Open Redirect in `/login` (High) — CWE-601

Location: `app.py`, lines 92–93 **Detected by:** Semgrep only (`open-redirect`) — *Bandit missed this*

`redirect(request.args.get('next'))` blindly redirects users to any URL after login, enabling phishing redirects from a trusted domain.

Remediation: Validate that `next` is a relative path or that its `netloc` matches the application's own host.

F-09 — Missing CSRF Protection on `/note/new` (High) — CWE-352

Location: `app.py`, lines 124–132 **Detected by:** Semgrep (`django-no-csrf-token`); also confirmed in manual review

State-changing POST endpoint validates no CSRF token. Any external site can forge note-creation requests from authenticated users.

Remediation: Install Flask-WTF and use `CSRFProtect(app)`. Add `{{ csrf_token() }}` to all forms.

---

4. Medium & Low-Severity Findings (Summary)

ID	Vulnerability	CWE	Detected by	Severity
F-10	Insecure session cookies (no Secure/HttpOnly/SameSite)	CWE-614, CWE-1004	Manual	Medium
F-11	Hardcoded `SECRET_KEY`	CWE-798	Bandit `B105`, Semgrep	Medium
F-12	Hardcoded API key `ADMIN_API_KEY`	CWE-798	Manual	Medium
F-13	Admin endpoint discloses key in response	CWE-200	Manual	Medium
F-14	Flask debug mode enabled (`debug=True`)	CWE-489	Bandit `B201`, Semgrep	Low

F-15	Service bound to `0.0.0.0`	CWE-668	Bandit `B104`, Semgrep	Low

A path-traversal vector also exists in `/download` via the `file` parameter (`os.path.join('docs', filename)` with no sanitization, allowing `..` traversal) — flagged manually; subsumed by F-04/F-05 class of input-validation failures.

---

5. Dependency Vulnerabilities

`pip-audit` identified **27 known CVEs across 5 packages** in `requirements.txt`:

Package	Pinned Version	CVE Count	Highest Severity Issue
`werkzeug`	2.1.2	12	Debugger PIN bypass, resource exhaustion
`urllib3`	1.26.20	5	Redirect handling, request-smuggling vectors
`requests`	2.25.1	5	Cert verification bypass, proxy auth leak
`idna`	2.10	3	DoS via crafted hostnames
`flask`	2.0.1	2	Cookie parsing issues

Remediation: Upgrade all pinned versions to current releases (`flask>=3.1.3`, `werkzeug>=3.1.6`, `requests>=2.33.0`, `urllib3>=2.7.0`, `idna>=3.15`). Adopt automated dependency scanning in CI (Dependabot, Renovate, or `pip-audit` as a pre-merge check).

---

6. General Recommendations & Secure Coding Best Practices

1. **Never concatenate user input into SQL, shell commands, file paths, URLs, or HTML.** Use parameterized queries, argument lists, allowlists, and template auto-escaping.
2. **Validate all input at the trust boundary** — type, length, format, and range.
3. **Store secrets outside the codebase** — use environment variables or a secret manager. Rotate the `SECRET_KEY` and any leaked API keys immediately.
4. **Hash passwords with a memory-hard, salted algorithm** — `bcrypt`, `argon2`, or `scrypt`. Never MD5 or SHA-1.
5. **Enable CSRF protection on every state-changing request** (Flask-WTF makes this one line of code).
6. **Set secure cookie flags:** `SESSION_COOKIE_SECURE=True`, `HTTPONLY=True`, `SAMESITE='Lax'`.
7. **Enforce authorization on every object access**, not just authentication. Every query touching user data must include `WHERE owner = ?`.
8. **Never run `debug=True` in production**, and never bind directly to `0.0.0.0` without a firewall/reverse proxy in front.
9. **Keep dependencies patched.** Integrate `pip-audit` (or equivalent) into CI to fail builds on known CVEs.
10. **Adopt a security review checklist** (OWASP ASVS Level 2 is a reasonable baseline) for every PR.

---

7. Conclusion

The VulnNotes audit demonstrates that **layered analysis — multiple SAST tools plus manual review — is essential**. Each tool found vulnerabilities the others missed:

- **Bandit** excelled at language-level issues (weak crypto, dangerous APIs).
- **Semgrep** caught framework-aware bugs Bandit cannot see (SSRF, open redirect, missing CSRF).
- **pip-audit** uncovered an entire risk category (vulnerable dependencies) that code scanning ignores.
- **Manual review** found the IDOR — a pure logic flaw no scanner can detect.

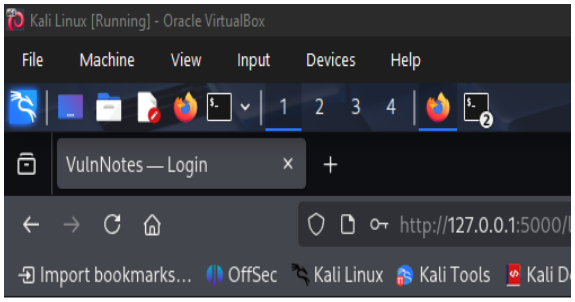
```
┌──(venv)─(douaa㉿kali)-[~/cybersec-internship/task-3-secure-code-review/vulnerable-app]
└─$ python app.py
 * Serving Flask app 'app' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: on
 * Running on all addresses (0.0.0.0)
   WARNING: This is a development server. Do not use it in a production deployment.
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.1.3:5000 (Press CTRL+C to quit)
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 797-397-060
127.0.0.1 - - [22/May/2026 15:13:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/May/2026 15:13:15] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [22/May/2026 15:14:23] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [22/May/2026 15:16:13] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [22/May/2026 15:16:13] "GET /notes HTTP/1.1" 200 -
```

```
Code scanned:
        Total lines of code: 145
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 3
                Medium: 7
                High: 5
        Total issues (by confidence):
                Undefined: 0
                Low: 2
                Medium: 4
                High: 9
Files skipped (0):
```

```
┌──(venv)─(douaa㉿kali)-[~/cybersec-internship/task-3-secure-code-review/vulnerable-app]
└─$ grep "Issue:" ../scans/bandit-report.txt
>> Issue: [B403:blacklist] Consider possible security implications associated with pickle module.
>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'super_secret_key_123'
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
>> Issue: [B704:markupsafe_markup_xss] Potential XSS with ``markupsafe.Markup`` detected. Do not use ``Markup`` on untrusted data.
>> Issue: [B704:markupsafe_markup_xss] Potential XSS with ``markupsafe.Markup`` detected. Do not use ``Markup`` on untrusted data.
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
>> Issue: [B602:subprocess_popen_with_shell_equals_true] subprocess call with shell=True identified, security issue.
>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible security issue.
>> Issue: [B201:flask_debug_true] A Flask app appears to be run with debug=True, which exposes the Werkzeug debugger and allows the execution of arbitrary code.
>> Issue: [B104:hardcoded_bind_all_interfaces] Possible binding to all interfaces.
```

```
┌──(venv)─(douaa㉿kali)-[~/cybersec-internship/task-3-secure-code-review/vulnerable-app]
└─$ tail -30 ../scans/bandit-report.txt
182         app.run(host='0.0.0.0', port=5000, debug=True)

--------------------------------------------------
>> Issue: [B104:hardcoded_bind_all_interfaces] Possible binding to all interfaces.
   Severity: Medium   Confidence: Medium
   CWE: CWE-605 (https://cwe.mitre.org/data/definitions/605.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b104_hardcoded_bind_all_interfaces.html
   Location: ./app.py:182:17
181         init_db()
182         app.run(host='0.0.0.0', port=5000, debug=True)

--------------------------------------------------

Code scanned:
        Total lines of code: 145
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 3
                Medium: 7
                High: 5
        Total issues (by confidence):
                Undefined: 0
                Low: 2
                Medium: 4
                High: 9
Files skipped (0):
```

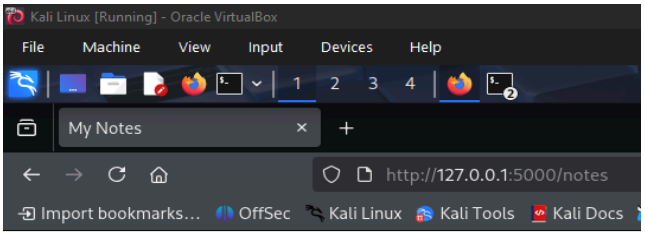
```
┌──(venv)─(douaa㉿kali)-[~/cybersec-internship/task-3-secure-code-review/vulnerable-app]
└─$ semgrep --config=auto . --exclude venv --text -o ../scans/semgrep-report.txt

┌─── ○○○ ────┐
│ Semgrep CLI │
└─────────────┘

Scanning 6 files (only git-tracked) with:

✓ Semgrep OSS
  ✓ Basic security coverage for first-party code vulnerabilities.

✘ Semgrep Code (SAST)
  ✘ Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.

✘ Semgrep Supply Chain (SCA)
  ✘ Find and fix the reachable vulnerabilities in your OSS dependencies.

💎 Get started with all Semgrep products via `semgrep login`.
✨ Learn more at https://sg.run/cloud.

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100% 0:00:00


┌──────────────┐
│ Scan Summary │
└──────────────┘
✓ Scan completed successfully.
 • Findings: 31 (31 blocking)
 • Rules run: 304
 • Targets scanned: 6
 • Parsed lines: ~100.0%
```



Login

Username: alice
Password: ••••••••••••
Login

Home



Notes

Search title... Search

- **Shopping list**: milk, bread, eggs view
- **Shopping list**: milk, bread, eggs view

New note

Title
Body
Save